

COMP101

Introduction to Programming

2025-26

Lecture-12

Selection Boolean

Boolean Type

- There are several types of data ('data types') in a programming language: Some languages have more data types than others
 - depends on how the program has been designed
 - 'Usual' data types are integer, float, string, Boolean (bool)
- Boolean types always evaluate True or False
 - They use 'comparison operators' and 'Boolean operators'
- comparison operators: == != or <> > < >= <=
- Boolean operators: and, or, not
 - used to make 'compound conditions'

Comparison Operators

- Comparison operators used in a selection statement always evaluate as True or False

Comparison Operator	Description	e.g.
==	equal to	2 == 2 evaluates True 2 == 3 evaluates False
!= or <>	not equal to	2 != 3 evaluates True 2 != 2 evaluates False
>	greater than	2 > 1 evaluates True 2 > 3 evaluates False
<	less than	2 < 3 evaluates True 2 < 1 evaluates False
> =	greater than or equal to	2 >= 2 evaluates True 2 >= 3 evaluates False
<=	less than or equal to	2 <= 2 evaluates True 2 <= 1 evaluates False

Comparison Operators: Opposites

- The operators have opposites of each other

Operator	e.g.	Opposite	Opposite e.g.
==	2 == 2 evaluates T	not or (!=)	2 != 2 evaluates F
>	2 > 1 evaluates T	< =	2 <= 1 evaluates F
<	2 < 3 evaluates T	> =	2 >= 3 evaluates F

Boolean & Comparison Operators: Precedence

- The operators fit into BODMAS

Precedence Table
() highest precedence
**
* / %
+ -
<, <=, >, >=, ==, !=
not
and
or (lowest precedence)

Boolean Operators: and

- Can evaluate two different inputs (a, b)
 - It is a 'binary operator'
- 'and' evaluates for a is True and b is True
 - if executes when both tests are True
 - else executes when either one of them is false

Truth Table

a	b	a and b
T	T	T
T	F	F
F	T	F
F	F	F

When there is an 'F' in it
the result will be False

Boolean Operators: and

- Can evaluate a single input (a)
 - It is a 'unary operator'
- if (a > 3 and a < 300) #test a range limit 4 to 299
 - if executes when a is in range
else executes when a is outside the range

Check the 'range limits': Value 3 and value 300 are 'outside the range'

When we use (a > 3 and a < 300) the range will be 4 to 299

When we use (a >= 3 and a <= 300) the range will be 3 to 300

- if (a > 3 and < 300) #gives an error – 'a' is missing in front of <300

Boolean Operators: and

“Assignment pass is in range 50 to 64”

numbers not match the spec

```
if (mark >=50 and mark <65)  
    print ('pass')
```

#numbers not match, logic not clear

```
if (50 >= mark <65)  
    print ('pass')
```

Both statements work, but can lead to confusion:
They are testing the range with number (65) not actually used in the specification.
The alternative also needs more thinking about what it means

#better to use

```
if (mark >= 50 and mark <= 64)  
    print ('pass')
```

Keep it simple:
Read the specification – it uses 50 to 64 - use these in the test

Boolean Operators: 'and' - compound expression

- Avoid the following for selection tests
 - can get very long, complicated and confusing
- 'Compound Expression'
 - several expressions joined together
if ((a > 3 and b < 300) and c = 200) and d = 50

They always read left-to-right: You can see this is getting complicated and hard to debug
e.g. would the test pass for any of the following individual var values?
When b=300?, when b=299?, when a=3? (answer = no, yes, no)

- When a compound expression gets too long use if-elif-else

Boolean Operators: or

- Can evaluate two different inputs (a, b)
 - It is a 'binary operator'
- 'or' evaluates for a is True or b is True
 - if executes when either a or b is True
 - else executes when both are false

Truth table

a	b	a or b
T	T	T
T	F	T
F	T	T
F	F	F

When there is an 'T' in it
the result will be True

Boolean Operators: or

- Can evaluate a single input (a)
 - It is a 'unary operator'
- if (a < 0 or a > 120)
 - Only one of the tests needs to evaluate True
 - If executes when the test is negative or >120
 - True for a < 0 hence is True for negative numbers from -1 down
 - True for a > 120 hence is True for positive numbers from 121 up
- if (a < 0 or > 120) #gives an error – 'a' is missing in front of >120
- Avoid complicated 'Compound expressions'
 - if ((a > 3 or b < 300) or c = 200) or d = 50
 - better to use if-elif-else instead

Boolean Operators: not

- Evaluates a single input (a)
 - It is a 'unary operator' – it doesn't use two inputs
- If you HAVE to use it, use it sparingly
 - can be useful but hard to understand
 - may be easier to test the positive condition (easier to read)

Truth table

a	not a
T	F
F	T

Boolean Operators: not

#Assignment pass mark at 40 or over using 'not'

```
if not mark < 40:  
    print ("pass")  
else:  
    print ("fail")
```

This reads as: 'if mark is not less than 40 then it must be 40 or above'
We are making readability and maintenance harder than it needs to be.
It is 'reversing the expression'

#Assignment pass mark at 40 or over – without 'not'

```
if mark >= 40:  
    print ("pass")  
else:  
    print ("fail")
```

Equivalent code written in the positive – easier to understand

Write the test in the positive first and if the positive looks confusing,
then try writing it with 'not'

Comparison Operators: not

#test i/p using 'not'

```
x = int(input("Enter a number: "))
```

```
if not x == 5:                #Note: syntax error if coded as if x not == 5
    print ("x is not 5")
```

#equivalent code – easier to understand

```
x = int(input("Enter a number: "))
```

```
if x != 5:                    #!= means 'not equal to'
    print ("x is not 5")
```

Boolean Operators: Short-Circuit Evaluation

- 'and', 'or' are short cut ('short circuit') operators
- The interpreter can operate more efficiently by examining the first expression in relation to the truth tables
- and – (a and b)
 - if executes when both are True
 - When a is false, no need to test b (the 'second expression')
 - 'a and b' will be False when 'a' is False
- or – (a or b)
 - If executes when either one is true
 - If a is True, then no need to test b
 - 'a or b' will be True when 'a' is true

Summary

- Avoid complicated tests: May indicate flawed logic
 - See if there's a simpler way to do it
- Use 'not' sparingly: Instead, coding in the positive is easier to read and makes more sense (avoiding double-negatives)
- if – elif – else: When well constructed is better than a large and complicated compound expression

Python if-elif-else

```
#other languages using over-indentation
#this shows an if and its associated else – every if has an else
month = input("Enter a month number 1 to 6: ")
if (month == 1) then
    print ("31 days")
else:
    if (month == 2) then
        print ("2 days")
    else:
        if (month == 3) then
            print ("31 days")
        else:
            if (month == 4) then
                print ("30 days")
            else:
                if (month == 5) then
                    print ("31 days")
                else:
                    if (month == 6):
                        print ("30 days")
```

Could use this as preliminary design – sort of pseudocode – to establish the logic

```
month = int(input("Enter a month number 1 to 6: "))
if (month==1):
    print ("31 days")

elif (month == 2):
    print ("28 days")

elif (month == 3):
    print ("31 days")

elif (month == 4):
    print ("30 days")

elif (month == 5):
    print ("31 days")

else:
    print ("month == 30 days") #confusing?
```

One if, one else , several
elifs – one elif for each
variable to be tested

Python if-elif-else

```
month = int(input("Enter a month number 1 to 6: "))
if (month==1):
    print ("31 days")

elif (month == 2):
    print ("28 days")

elif (month == 3):
    print ("31 days")

elif (month == 4):
    print ("30 days")

elif (month == 5):
    print ("31 days")

else:
    print ("Month 6 has 30 days")
```

There are 6 vars here (1 – 6):
The single if and the single else have handled 2 of the vars (1 and 6) leaving 4 vars to deal with (2,3,4,5) using the elifs.

We use the single else to 'capture the exception':
By eliminating 1,2,3,4,5 to be False the only number left is 6 – no test is needed as it must test True by elimination of the other vars.
The code has a weak structure

#robust code using Booleans

```
month = int(input("Enter a month number 1 to 6: "))

if (month==1 or month==3 or month==5):
    print ("31 days")

elif (month==4 or month==6):
    print ("30 days")

else:
    print ("28 days")
```

The single else has 'captured the exception'.
Numbers 1,3,4,5,6 have been eliminated as False.
The only number left is 2 – no test is needed – it tests True